Chat bot used: ChatGPT

Questions generated:

1. Deep Neural Networks: Explain the architecture and learning process of a deep neural network. Discuss how forward propagation, backpropagation, activation functions, and optimization algorithms work together to enable a neural network to learn complex patterns.
2. Loss Functions: Compare and contrast different loss functions used in neural networks, such as Mean Squared Error, Binary Cross-Entropy, and Categorical Cross-Entropy. Explain how the choice of loss function affects model training and performance, and provide examples of situations where each would be appropriate.
3. Convolutional Neural Networks: Explain how convolutional neural networks (CNNs) are designed for image-related tasks. Discuss the roles of convolutional layers, filters/kernels, activation functions, pooling layers, and fully connected layers, and explain why CNNs are generally effective at extracting features from images.

First answers:

1. There are K number of hidden layers, with the number of layers being referred to as the depth, and the number of hidden units in each layer as the width. Focusing on just the first layer, you have the input of however many parameters, and those inputs are passed through an activation function, where the output is the hidden units of the next layer. Each following layer follows the same process, but using the functions from the previous layer as the inputs for those units. This process is also what forward propagation is, that movement from the input to the output through the hidden layers. I'm not sure about backpropagation of optimization algorithms.

2. First with Mean Squared Error, you use that with a univariate regression model. Binary Cross-Entopy loss is used for binary classification. And Categorical Cross-Entropy is used for Multiclass Classification. For the Binary Cross-Entropy loss, they see that variance between how confident the model was in the right answer, and essentially how far off that number was (since ideally it should be as close to 1 as possible). The idea is similar for Categorical Cross-Entropy, with the main difference being that there are K numbers of categories rather than just two. This means that you have to choose the best suited loss function for two factors. 1) how you will be categorizing your predicted output, and 2) what that output looks like (univariate vs multivariate). I'm not sure how it may directly affect the training process, but since the outcomes are different, it changes what you are able to do with your prediction.

3. Since with images you would have to compare individual pixels, to compare all of the would be too high of a workload, especially if you have a fully connected network. The concept that Convolutional networks use is that ideally, depending on the subject pictures, nearby pixels should relatively be part of/representing the same thing/object. So, to lighten this workload, they don't use every pixel. Additionally, instead of having fully connected layers, where every hidden unit from the layer before connects to every hidden unit in the layer after it, they may have a set stride and only input into some of the units. The stride is how many units over it will move (meaning not every hidden unit in one layer will have a corresponding unit in the next) dilation, which determines which hidden units will input into the next hidden unit. Say if we have h5 feeding into h5', but with a dilation of two, then h3 and h7 (assuming they exist) will also input into h5'.

Critique one:
"*Questions generated:*
Deep Neural Networks: Explain the architecture and learning process of a deep neural network. Discuss how forward propagation, backpropagation, activation functions, and optimization algorithms work together to enable a neural network to learn complex patterns.
Loss Functions: Compare and contrast different loss functions used in neural networks, such as Mean Squared Error, Binary Cross-Entropy, and Categorical Cross-Entropy. Explain how the choice of loss function affects model training and performance, and provide examples of situations where each would be appropriate.
Convolutional Neural Networks: Explain how convolutional neural networks (CNNs) are designed for image-related tasks. Discuss the roles of convolutional layers, filters/kernels, activation functions, pooling layers, and fully connected layers, and explain why CNNs are generally effective at extracting features from images.

First answers:
There are K number of hidden layers, with the number of layers being referred to as the depth, and the number of hidden units in each layer as the width. Focusing on just the first layer, you have the input of however many parameters, and those inputs are passed through an activation function, where the output is the hidden units of the next layer. Each following layer follows the same process, but using the functions from the previous layer as the inputs for those units. This process is also what forward propagation is, that movement from the input to the output through the hidden layers. I'm not sure about backpropagation of optimization algorithms.

First with Mean Squared Error, you use that with a univariate regression model. Binary Cross-Entopy loss is used for binary classification. And Categorical Cross-Entropy is used for Multiclass Classification. For the Binary Cross-Entropy loss, they see that variance between how

confident the model was in the right answer, and essentially how far off that number was (since ideally it should be as close to 1 as possible). The idea is similar for Categorical Cross-Entropy, with the main difference being that there are K numbers of categories rather than just two. This means that you have to choose the best suited loss function for two factors. 1) how you will be categorizing your predicted output, and 2) what that output looks like (univariate vs multivariate). I'm not sure how it may directly affect the training process, but since the outcomes are different, it changes what you are able to do with your prediction.

Since with images you would have to compare individual pixels, to compare all of the would be too high of a workload, especially if you have a fully connected network. The concept that Convolutional networks use is that ideally, depending on the subject pictures, nearby pixels should relatively be part of/representing the same thing/object. So, to lighten this workload, they don't use every pixel. Additionally, instead of having fully connected layers, where every hidden unit from the layer before connects to every hidden unit in the layer after it, they may have a set stride and only input into some of the units. The stride is how many units over it will move (meaning not every hidden unit in one layer will have a corresponding unit in the next) dilation, which determines which hidden units will input into the next hidden unit. Say if we have h5 feeding into h5', but with a dilation of two, then h3 and h7 (assuming they exist) will also input into h5'.

critique my answers"

Updated answers:

1. There are K number of hidden layers, with the number of layers being referred to as the depth, and the number of hidden units in each layer as the width. Focusing on just the first layer, you have the input of however many parameters, and those inputs are transformed by the corresponding weight and bias, then that is passed through an activation function, where the output *becomes the input* of the next layer. Each following layer follows the same process, but using the functions from the previous layer as the inputs for those units. This process is also what forward propagation is, that movement from the input to the output through the hidden layers. Backpropagation is when you work back to see how the chosen weights and biases contributed to the loss. It is then that you use that to find the *loss gradient*. From there, you can adjust those variables using the learning rate to try and find the *best parameter values* that help *minimize the loss*. Specifically, it is the optimization algorithm that updates the values of the weights and biases to try and find that minimum loss. Finally, the reason that the activation functions are so important. They expand the data to take it from being just multiple linear layers,

adding nonlinearity, and allowing the model to understand more *complex relationships in the data*.

- Some of my explanations still weren't quite right, specifically with my word choice. Especially since a lot of my wording was in absolutes, but technically you're not looking for the absolute minimum loss, for example, but just the smallest that you can reasonably find.

2. First with Mean Squared Error, you use that with regression problems involving continuous numerical values. Binary Cross-Entopy loss is used for binary classification. And Categorical Cross-Entropy is used for Multiclass Classification. For the Binary Cross-Entropy loss, it sees how confident the model was in its decision, and whether it was right or wrong. It uses this to penalize answers that were wrong, and strongly penalize answers that were wrong with very high confidence. The idea for Categorical Cross-Entropy, *the loss looks at the probability the model assigned to the correct class. It gives a low loss when the model assigns high probability to the correct class and a high loss when it assigns low probability to the correct class*. This means that you have to choose the best suited loss function for two factors. 1) how you will be categorizing your predicted output, and 2) what that output looks like (univariate vs multivariate). In regards to the training process, the loss function directly contributes to making the gradient used in backpropagation, which will tell us how to update the weight and bias values to try to minimize our loss.
    a. MSE: commonly used when predicting continuous numerical values.
    b. BCE: commonly used when predicting a binary outcome, usually with a sigmoid output.
    c. CCE: commonly used when predicting one class among multiple mutually exclusive classes, usually with a softmax output.

    - My answer seemed mostly correct for this, just that I needed to fix my definition for Categorical Cross-Entropy

3. Since with images you would have to compare individual pixels, to compare all of them would be too high of a workload, especially if you have a fully connected network. The concept that Convolutional networks use is that ideally, depending on the subject pictures, nearby pixels should relatively be part of/representing the same thing/object. So, to lighten this workload, they don't fully connect all input pixels to every neuron. Instead they use the filter to examine regions in an image that will connect to a neuron.

Additionally, instead of having fully connected layers, where every hidden unit from the layer before connects to every hidden unit in the layer after it, they may have a set stride and only input into some of the units. The stride is how many *pixels* over the filter will move (meaning how far the area of input in the image will be from the last). With that, a filter is a small matrix of learnable weights that slides across the image (the input area). During training, the network learns filters that detect useful patterns such as edges, textures, shapes, and eventually more complex structures. Dilation allows the model to span a larger part of the image without taking in more parameters (so like reading in every other or every third pixel rather than the pixels that are next to each other). Much like the other networks, it is between those hidden layers that an activation function is used to create that nonlinearity, adding to the relational understanding of the pixels/image. *Pooling reduces the spatial dimensions of feature maps, which decreases computation and the number of parameters while retaining important information about the detected features.* It is after all this initial processing that we can go back to fully connected layers, since we're working with less parameters, and do that same classification process like with the other architectures. It isn't always necessary, but helps if you are trying to classify an image in some capacity.

- It wanted me to go more in depth about why pooling was important, as well as fix some of my exact language, since it wasn't quite 100% correct. For example, saying that stride moves in pixels not units, since that wording is a bit misleading/confusing.

I honestly learned a lot from this, and because it forced me to make sure that my answers were 100% correct, it means that I now understand the concepts that much more. It definitely helped just being honest from the get go, writing down what I for sure did or didn't know (or at least what I thought I did and didn't know) so that way it could slowly get me to the absolute correct answer. It's definitely a cycle that you could continue to do until you have a much more confident and correct answer/understanding. There were a few times with the answers that ChatGPT gave me/things it had me change that I specifically asked it to expand on at the moment. I feel like this helped really ensure that my understanding of the concepts was correct.